

Leetcode

1. Strings

1. convert string to list: `list(s)`, where `s` is a string.
2. convert list to string: `".join(s)`, where `s` is a list.
3. Ascii format: can use `"b">"a"`: True, `"a">"b"`: False.

2. •

1. list: an ordered sequence (repeats allowed).
2. hashset: a collection of distinct objects where order does not matter.
3. multiset: a collection of objects (repeats allowed) where order does not matter, for example Counter in Python collections.
4. ordered set: a collection of distinct objects where order matters.

3. ordered list is also called **string**, a *word*, or a *sequence*.

4. A list is also called a *sample* or a *selection*, especially in probability and statistics.

5. Lists are sometimes called vectors and the elements components.

6. substring, subsequence, subarray

1. **substring** is a *continuous* part or subpart of a string.
2. whereas, **subsequence** is the part of a string or sequence, that might be continuous or not but the *order* of the elements is maintained.
3. Example:
 1. `s="pwwkew"`, `"wke"` is a substring, whereas, `"pwke"` is a subsequence not a substring.
4. subarray is a contiguous sequence in an array
 1. Example: `{1,2,3,4}`
 2. subarray: `{1,2}`, `{1,2,3}`
 3. subsequence: need not be contiguous, but maintains order: `{1,2,4}`
 4. subset: same as subsequence except it has empty set: `{1,3}`, `{}`

7. Patterns:

1. Two pointers
2. Fast and slow pointers
3. Sliding window
4. Merge intervals
5. Cyclic sort

8. Recursion:

1. Write out the code for a recursive function that sums elements in a list.

1. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 11:53:43 2026

@author: reina
"""

"""
A recursive sum function.
"""

def sum_recursive(x):
    """
    x: list[int]
    """
    if not x:
        return 0
    if len(x)==1:
        return x[0]
    return x[0]+sum_recursive(x[1:])

if __name__ == '__main__':
    x = [12,2,30,9]
    res=sum_recursive(x)
    print(res)
```

2. Write a recursive function to count the number of items in a list.
3. Write a recursive function to find the maximum number in a list.

1. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 13:19:17 2026

@author: reina
"""

def max_recursive(x):
    if len(x)==2:
        return x[0] if x[0]>x[1] else x[1]
    if len(x)==1:
        return x[0]
    else:
        l,r=0,len(x)-1
        mid=l+(r-l+1)//2
        left=x[:mid]
        right=x[mid:]
        max_left=max_recursive(left)
        max_right=max_recursive(right)
        return max_left if max_left>max_right else max_right

if __name__ == '__main__':
    x=[2,5,7,1,3,5,6]
    # x=[2,-20,21,3,11]
    res=max_recursive(x)
    print(res)
```

4. Binary search is a D&C algorithm, too. Can you come up with a base case and recursive case for binary search?

1. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 15:14:48 2026

@author: reina
"""

def binary_search(x,target):
    l,r=0,len(x)-1
    def helper(l,r):
        if l<r:
            mid=l+(r-l+1)//2
            print(l,mid,r)
            if x[mid]==target:
                return mid
            if target<x[mid]:
                return helper(l,mid-1)
            else:
                return helper(mid,r)
        else:
            return -1
    return helper(l,r)

if __name__ == '__main__':
    x=[2,3,5,7,11,13]
    x=[2]
    target=11
    target=3
    res=binary_search(x,target)
    print(res)
```

5. Quicksort vs mergesort

1. Quicksort is used more in practice. Quicksort has average time complexity of $O(n \log(n))$ and worst case complexity of $O(n^2)$. Quicksort is an in-place algorithm.
2. mergesort is always $O(n \log(n))$. However, the simple implementation of mergesort is not an in-place algorithm, we need an auxiliary array to merge the two sorted arrays, hence it has $O(n)$ space complexity.

6. mergesort:

1. Code:

```
# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 13:34:49 2026

@author: reina
"""

def mergesort(x):
    if len(x)<=1:
        return x
    else:
        l,r=0,len(x)-1
        mid=l+(r-l+1)//2
        # mid=len(x)//2
```

```

    left=mergesort(x[:mid])
    right=mergesort(x[mid:])
    #merge
    res=[]
    i,j=0,0
    while i<len(left) and j<len(right):
        if left[i]<right[j]:
            res.append(left[i])
            i+=1
        else:
            res.append(right[j])
            j+=1
    if i==len(left):
        res.extend(right[j:])
    elif j==len(right):
        res.extend(left[i:])
    return res

if __name__ == '__main__':
    x=[1,11,3,5,-1,13,2]
    # x=[1,11]
    res=mergesort(x)
    print(res)

```

2. Quicksort

References:

1. Grokking the Coding Interview, Design Gurus.
2. Grokking Algorithms, Aditya Bhargava.